

Snowflake CoCo CLI Hackathon — Research & Strategy

Sep 29, 2026 · @Create a new relic monitor to track successful login process. Th

TL;DR

Build CashlessIQ on Problem Statement #4 (Patient and Member 360) and submit the prototype by Sunday 4 Oct, 6 PM IST. There is no separate idea round: the prototype submission (closing 4 Oct, 11:59 PM IST) is the first cut, so the MVP must work end to end in five days.

- **Why #4:** zero public competitor repos found on this track, versus 6+ on Supply Chain and 2 each on Fraud and Customer 360.
- **What it is:** a cashless pre-authorisation copilot. A hospital request PDF goes in; a Cortex Agent checks the member 360 and the policy wording, and drafts an approve / deduct / query / refer decision with a citation on every line — inside IRDAI's one-hour cashless window.
- **What you'll learn on Snowflake:** Cortex AI functions, Cortex Search, semantic views, Cortex Agents, Snowpark Python, Streamlit in Snowflake, masking policies — all built through CoCo CLI, which is mandatory under the rules.
- **Submission must include:** idea, working prototype, a presentation deck, and a documented GitHub repo; finalists demo live on 27–30 Oct.
- **Do tonight:** confirm the \$400 account came from the Contest Site link and that `cortex` connects to it; confirm teammates are registered before registration closes on 30 Sep.

Hackathon at a glance

The prototype window closes on **4 Oct 2026** — five days from today — and there is no separate idea round listed, so the prototype submission *is* the first cut. Technical Execution carries the most weight at 40%.

Item	Detail
Name	Snowflake CoCo CLI Hackathon 2026 — GCC Edition
Organizer / sponsor	Hack2skill, for Snowflake

Item	Detail
Who	India GCC practitioners (software, data, AI/ML, backend engineers), age 18+
Team size	1-4 (solo allowed)
Format	Online; finale is an online demo day
Fee	Free
Brief	Build enterprise-ready, AI-native apps on enterprise data using Snowflake CoCo CLI — "not just prototypes"
Support	cococlihackgcc-support@hack2skill.com · Snowflake Discourse
Rules	Terms and Conditions

Timeline

Date	Milestone
15 Jun 2026	Registrations open
6 Aug 2026	Problem statement explainer session
12 Aug 2026	Workshop 1 — CoCo CLI starter (recorded, on demand)
19 Aug 2026	Workshop 2 — CoCo CLI hands-on (recorded, on demand)
13 Sep – 4 Oct 2026	Prototype submission phase
30 Sep 2026	Registration closes
5 – 22 Oct 2026	Prototype evaluation
23 Oct 2026	Final shortlist announced
26 Oct 2026	Induction session
27 – 30 Oct 2026	Grand finale demo days

How you are judged

Criterion	Weight	What it means in practice
Technical Execution	40%	Depth of Snowflake usage, working code, sound architecture
Real-World Relevance	30%	A believable enterprise user and a pain the problem statement names
Solution Completeness	30%	End-to-end flow that works in a demo, not a slideware stub

Prizes (USD 10,000 pool, per team)

Place	Prize
Winner	\$4,300 / ₹4,00,000
1st runner-up	\$2,200 / ₹2,00,000
2nd runner-up	\$1,590 / ₹1,50,000
Consolation	\$530 / ₹50,000

Open question: the page does not list the exact submission artifacts (repo, deck, video?). Check the submission form on your Hack2skill dashboard today.

What CoCo CLI is

CoCo is Snowflake's AI coding agent — the product formerly called **Cortex Code**, renamed at Snowflake Summit on 2 June 2026 ([Atlan explainer](#)). The CLI is an agentic shell in your terminal: you type what you want in English, and it writes and runs SQL, Python, bash and git against your Snowflake account under your role's RBAC ([Snowflake docs](#)). Think of it as Claude Code, but with Snowflake-native tools and skills built in.

Important for you: CoCo is the *build tool*, not the product judges score. Judges score the app you ship on Snowflake. Use CoCo visibly to build it — and say so in the demo — but the win comes from what the app does.

Setup in 10 minutes

1. Install Snowflake CLI (`snow`) — CoCo reuses its `~/ .snowflake/connections.toml` .

2. Install CoCo CLI: `curl -LsS https://ai.snowflake.com/static/cc-scripts/install.sh | sh` (macOS/Linux/WSL) or `irm https://ai.snowflake.com/static/cc-scripts/install.ps1 | iex` (Windows PowerShell).
3. Run `cortex` ; the wizard picks or creates a connection.
4. Your user needs the `SNOWFLAKE.CORTEX_USER` (or `CORTEX_AGENT_USER`) database role; it is granted via PUBLIC by default.
5. First prompt: "What can I do with Cortex Code?"

Source: [CoCo CLI docs](#).

Account check (do this today): CoCo CLI does not run on standard trials from trial.snowflake.com. The rules (§4.3) say the contest gives each registrant a trial account with \$400 credit through a sign-up link on the Contest Site. Make sure your \$400 account came from that link, then run `cortex` against it once to confirm CoCo works.

Features worth knowing

Feature	What it gives you
Models	Claude, OpenAI GPT, Gemini, Grok via Cortex; <code>/model</code> to switch; Auto / Auto Intelligent / Auto Efficient presets
Plan mode	Confirms each action before running — use it on anything that creates objects
Headless mode	<code>cortex --print</code> for scripted runs (good for a reproducible <code>setup.sh</code> in your repo)
Code mode	<code>cortex --mode code</code> — lean coding toolset, fewer tokens per turn, no Snowflake data tools (docs)
Skills	55+ bundled <code>SKILL.md</code> skills, auto-loaded by intent or via <code>/skill</code> ; custom ones live in <code>~/.snowflake/cortex/skills/</code>
MCP	<code>cortex mcp add <name> <url></code> to plug in external tools; config in <code>~/.snowflake/cortex/mcp.json</code> (docs)
Plugins	One package bundling skills, subagents, slash commands, hooks and MCP servers (docs)

Bundled skills that matter for this hackathon

From the [bundled skills reference](#):

Skill	Use it to
<code>agent-studio</code>	Create Cortex Agents, build semantic views with verified queries, evaluate agents, publish to Snowflake Intelligence
<code>ai-functions-pipeline-builder</code>	Turn a plain-English ask into an incremental pipeline of Dynamic Tables + Cortex AI functions over docs and tables (templates include Customer 360 and Structured Extraction)
<code>document-intelligence</code>	AI_EXTRACT, AI_CLASSIFY, AI_SENTIMENT, OCR, parsing on staged files
<code>search-optimization</code>	Build a Cortex Search service over PDFs, DOCX, audio, video on a stage
<code>cortex-ai-function-studio</code>	Build and evaluate custom AI functions on AI_COMPLETE; compare models on cost and quality
<code>developing-with-streamlit</code>	Build and theme a Streamlit in Snowflake app
<code>snowflake-apps</code>	Scaffold and deploy a Next.js app on Snowflake App Runtime (<code>snow app deploy</code>)
<code>machine-learning</code>	Train, register and serve models; forecasting and anomaly detection
<code>dynamic-tables</code> , <code>snowflake-tasks</code> , <code>alert</code> , <code>notification</code>	Keep data fresh and push alerts to Slack or email
<code>data-governance</code>	PII classification, masking policies, row access policies, access audit
<code>lineage</code> , <code>data-quality</code>	Lineage tracing and Data Metric Functions — useful for "trust" features
<code>skill-development</code>	Write your own CoCo skill — a strong differentiator (see recommendation)

Snowflake services you will use

The winning pattern on Snowflake right now is **Cortex Agent = semantic view (structured) + Cortex Search (documents) + custom tools**, wrapped in a Streamlit app. Learn these six in this order; skip the rest for this hackathon.

#	Service	What it does	How you touch it	Learn first
1	Cortex AI Functions	LLM calls as SQL: AI_COMPLETE, AI_EXTRACT, AI_CLASSIFY, AI_AGG, AI_SUMMARIZE_AGG, AI_TRANSCRIBE, AI_PARSE_DOCUMENT, AI_REDACT, AI_EMBED	<pre>SELECT AI_EXTRACT(...) FROM ...</pre>	AI_PARSE_DOCUMENT → AI_EXTRACT on a staged PDF
2	Cortex Search	Managed hybrid (vector + keyword) retrieval over text chunks — your RAG layer	<pre>CREATE CORTEX SEARCH SERVICE ... ON chunk</pre>	Chunking, attributes for filtering, citations
3	Semantic views	Schema object holding business entities, relationships, metrics, synonyms, verified queries	<pre>CREATE SEMANTIC VIEW or CoCo agent-studio skill</pre>	Metrics vs dimensions, verified queries
4	Cortex Agents	Orchestrator: plans, calls Analyst (SQL over semantic views), Search, custom tools (stored procs/UDFs), code execution, data-to-chart, MCP connectors	<pre>CREATE AGENT + agent REST API</pre>	Tool descriptions and orchestration instructions
5	Streamlit in Snowflake	Python UI hosted inside Snowflake; judges give it "special consideration"	<pre>snow streamlit deploy</pre>	Session, caching, chat UI

#	Service	What it does	How you touch it	Learn first
6	Snowpark Python	DataFrames, UDFs and stored procedures in Python (Java/Scala also supported)	<code>snow snowpark deploy</code>	Stored proc as an agent tool

Useful extras if time allows: **Dynamic Tables** (declarative incremental pipelines), **Tasks + Alerts + Notifications** (push an action to Slack/email), **masking and row access policies** (governance), **Snowflake Marketplace** (free public datasets — also a judging bonus), **Snowflake Intelligence / CoWork** (the no-code chat front end for your agent).

Three platform facts that change design choices

- **Cortex Analyst is folding into Agents.** On 28 Aug 2026 Snowflake recommended calling Cortex Analyst through Cortex Agents; semantic views carry over unchanged ([release note](#)). Build on Agents, not the old Analyst REST API.
- **Summit 2026 renames.** Cortex Code became CoCo, Snowflake Intelligence became **CoWork**, and **Cortex Sense** (a context layer, private preview) was announced ([Atlan](#)). Using current names in your deck signals you are up to date.
- **AI Credits are separate from platform credits.** AI functions, Agents, Search and CoWork bill in AI Credits per million tokens, rates varying by model ([Snowflake AI pricing](#)). Model choice is your biggest cost lever.

Judging criteria hidden in the rules

The public page shows a 30/40/30 rubric, but §9 of the [Terms](#) adds the concrete checklist judges apply:

1. Responsive to a problem statement, **including use of Cortex Code CLI** (mandatory).
2. Code in **Python, Java and/or Scala**.
3. Snowflake platform use is **required**.
4. **Special consideration** for Snowpark, Worksheets, Streamlit and/or **Snowflake Marketplace**.

The entry must include the idea, the prototype, a **presentation deck**, and a **GitHub repo with clear documentation** (§4.1, §4.5). Finalists demo **live** — pre-recorded demos need sponsor approval. You must list every dataset used and link its license (§4.3b).

The five problem statements, compared

Pick #4, Patient and Member 360. It has the fewest visible competitors, the best fit with Snowflake's hero stack (Agents + Search + semantic views), and a regulatory angle your identity/security background makes credible. Supply Chain is the most crowded track by far.

Competition below counts public GitHub repos I found that name this hackathon and the track — a signal of crowding, not a full census.

#	Problem statement	Public repos seen	Snowflake-stack fit	Data for 5 days	Room to stand out	Verdict
4	Patient/Member 360 + clinical or regulatory document copilot	0	Very high — structured 360 + cited document answers is exactly Agent + Search	Synthea (Apache-2.0 synthetic patients) + public policy/regulatory text	High — nobody visible; a decision workflow beats a chat-over-records demo	Recommended
3	Predictive Maintenance + OEE command center	1 (example)	Medium — leans on Snowpark ML more than Cortex	NASA C-MAPSS turbofan data	Medium — ML accuracy is hard to prove in a live demo	Backup pick
2	Customer 360 + Next Best Action	2 (NorthStar-CX , Workforce-Astra)	High — AI_TRANSCRIBE on calls + agent	Synthetic customers + call transcripts	Medium — NBA engines look alike	Possible
1	Risk, Fraud and Regulatory Intelligence copilot	2 (fraud-policy-copilot , ClauseTrace)	High	Synthetic transactions; RBI/AML docs	Low-medium — the most obvious fintech idea	Avoid
5	Supply Chain ontology + governed conversational analytics	6+ (ChainLoom , OntoFlake , rajanand , SupplyChainIQ , snowflake-forge , sasidhar)	Very high	Synthetic ERP data	Low — several strong, polished entries already (MCP servers, custom skills, Marketplace joins)	Avoid

What the visible competitors already do (your bar)

The strongest public entries already ship: a deployed Streamlit-in-Snowflake app, a semantic view with verified queries, Cortex Search citations, a custom CoCo skill in `.cortex/skills/`, a custom MCP server registered with CoCo, row-access and masking policies, a Marketplace dataset join, and a judge-facing demo script. **Treat that as table stakes, not a differentiator.** Your edge has to come from the use case and one or two features nobody else has.

One lesson from a repo in the earlier edition: judges could not log in with shared credentials because of a password mismatch ([ShirishAcharya](#)). Create a dedicated judge user and test it from a private window before submitting.

Recommended build: CashlessIQ

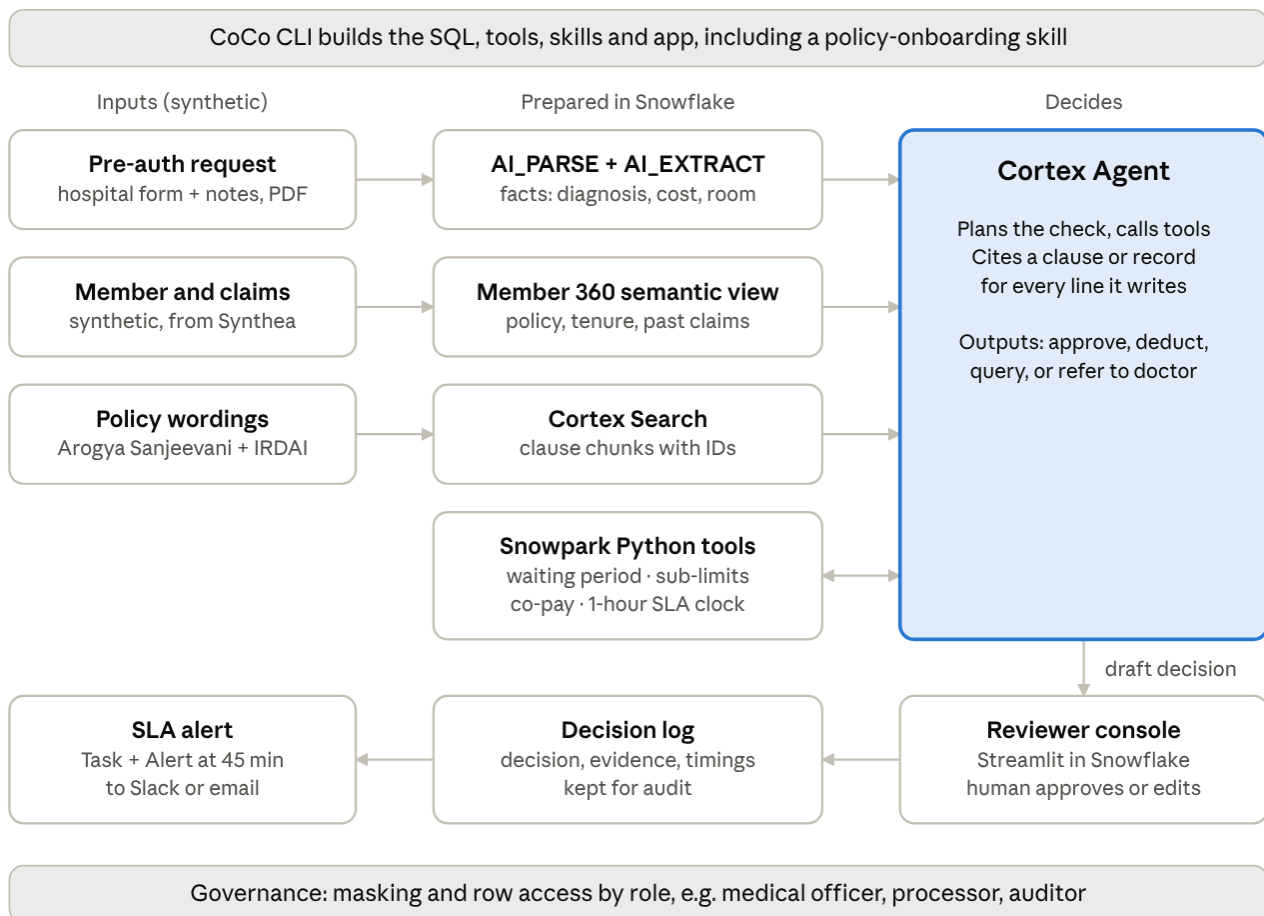
Build **CashlessIQ**, a cashless pre-authorisation copilot for health insurers and TPAs. It reads a hospital's cashless request, assembles the member 360, checks it against the policy wording, and drafts an *approve / approve with deductions / query / refer to doctor* decision in minutes — with every line citing a policy clause or a member record. It answers Problem Statement #4 (member 360 + regulatory document copilot with cited evidence) on fully synthetic member data.

Why this use case scores

- **A real regulatory clock.** IRDAI's Master Circular of 29 May 2024 requires a cashless authorisation decision within one hour of the request and final discharge authorisation within three hours; delay costs past three hours fall on the insurer ([Life Insurance International](#), [RTI wiki summary](#)). A measurable pain lands Real-World Relevance (30%).
- **Real, public rule documents.** Arogya Sanjeevani is IRDAI's standard health product, and insurers' wordings are published on [irdai.gov.in](#) — e.g. room rent capped at 2% of sum insured up to ₹5,000/day and cataract at 25% of sum insured or ₹40,000 per eye ([sample wording on IRDAI](#)). Real clauses make grounding and citations meaningful.
- **Empty lane.** No public repo on this track turned up; the crowd is in supply chain and fraud.
- **Your edge.** Role-based access, masking and audit trails are CIAM territory — you can build that layer better than most entrants.

Architecture

One agent turns a pre-auth request into a cited decision



CashlessIQ architecture · inputs → Snowflake prep → agent → reviewer

The agent never does arithmetic itself: Snowpark tools compute waiting periods, sub-limits and proportionate deductions, and the agent explains their output with citations.

Seven differentiators

1. **A decision, not a chat.** Output is a structured determination with line items, not free text over records.
2. **Deterministic math.** Tools compute every rupee; a judge can re-run any number.
3. **Evidence on every line.** Clause ID from Cortex Search, record ID from the semantic view, clickable in the console.
4. **SLA clock on screen.** Timer from request receipt; a Task + Alert escalates at 45 minutes.
5. **Role-aware governance.** Masking hides clinical notes from processors; row access limits each TPA to its own members.

6. **CoCo as a factory.** A custom `policy-onboarding` skill: point CoCo at a new policy PDF and it extracts sub-limits and waiting periods into the rules table, indexes the clauses, and writes test cases. Onboard a second product live in the demo.
7. **Measured accuracy.** A golden set of ~30 synthetic requests with expected outcomes and a scorecard in the app (use the `agent-studio` evaluation workflow).

Coverage of the judging checklist

Judging item	How CashlessIQ covers it
CoCo CLI use (mandatory)	Whole build driven through CoCo; custom skill in <code>.cortex/skills/</code> ; prompt log in <code>docs/coco-playbook.md</code>
Python / Java / Scala	Snowpark Python tools + Streamlit; optional Java UDF (e.g. ICD-10 code validation) to show range
Snowflake platform	Everything runs inside Snowflake — stage, AI functions, Search, Agent, app
Snowpark, Worksheets, Streamlit, Marketplace	Snowpark and Streamlit core; setup SQL as Worksheets; Marketplace as a stretch (see below)
Technical Execution (40%)	Agent + Search + semantic view + custom tools + governance + eval
Solution Completeness (30%)	End to end: PDF lands → decision logged → alert fired

Marketplace, honestly: the free [Snowflake Public Data](#) listing is mostly US data (CMS, NPES providers). Only add it if you build a US prior-auth mode (e.g. validating the requesting provider's NPI). Don't force an irrelevant join.

Three-minute demo script

1. A hospital request PDF lands on the stage; the SLA clock starts.
2. Case A: knee replacement, room above the cap → *approve with proportionate deduction*, each line cited.
3. Case B: member inside a waiting period for the condition → *query*, citing the clause and the enrolment date.
4. Switch role to auditor → clinical notes masked, decision log intact.
5. Run the CoCo `policy-onboarding` skill on a second policy; rerun Case A under it.

6. Show the accuracy scorecard on the golden set.

Backup pick: if healthcare feels too heavy, #3 Predictive Maintenance has only one visible competitor; use NASA C-MAPSS data, Snowpark ML for remaining-useful-life, and an agent that drafts work orders.

Five-day MVP build plan

You have five working days: tonight through Sunday 4 Oct, with submission closing at 11:59 PM IST. Aim to submit by **6 PM on Sunday** and keep the evening as buffer. Every day ends with something working end to end, even if thin.

Three platform gotchas to design around from day one

- **Streamlit runtime:** Cortex Agent APIs are not supported in Streamlit-in-Snowflake apps on the *warehouse* runtime ([Cortex Agents docs](#)). Create the app on the **container runtime** (GA since 9 Mar 2026) from the start.
- **Cross-region inference:** agents route model calls cross-region, and `CORTEX_ENABLED_CROSS_REGION` is disabled by default except in organisations created after 9 Mar 2026 ([get started guide](#)). Check it on day 0.
- **Default role, not session role:** Cortex Agents take permissions from the user's *default* role. For the role-based masking demo, create separate demo users (medical officer, processor, auditor), each with its own default role — switching roles in one session won't work.

Tuesday 29 Sep (tonight) — setup

- ☐ Confirm your \$400 account came from the Contest Site sign-up link; confirm teammates are registered (registration closes 30 Sep)
- ☐ Install `snow` + `cortex`; run `cortex` and ask it to list your databases
- ☐ As ACCOUNTADMIN: check cross-region inference; create a resource monitor; create an XS warehouse with 60-second auto-suspend
- ☐ Create GitHub repo: `sql/`, `tools/`, `app/`, `data/`, `.cortex/skills/`, `docs/`
- ☐ Download two Arogya Sanjeevani wordings from different insurers and the IRDAI health master circular

Wednesday 30 Sep — data

- ☐ Generate ~500 Synthea patients (CSV) and use CoCo to map them into `MEMBERS`, `POLICIES`, `ENROLMENTS`, `CONDITIONS`, `CLAIMS`

- ☐ Write a Python generator for ~30 synthetic pre-auth request PDFs (hospital form + short clinical note) with a `golden_outcomes.csv`: approve, deduct, query, refer
- ☐ Plant the tricky cases: room above cap, condition inside a waiting period, cataract over sub-limit, missing document
- ☐ Load PDFs to an internal stage; load tables

Thursday 1 Oct — understand documents and data

- ☐ `AI_PARSE_DOCUMENT` → `AI_EXTRACT` pipeline into `PREAUTH_FACTS` (diagnosis, procedure, estimated cost, room type, admission date)
- ☐ Chunk policy wordings by clause with stable clause IDs; create the Cortex Search service with product and version as filter attributes
- ☐ Build the `MEMBER_360` semantic view with CoCo's `agent-studio` skill; add 10 verified queries

Friday 2 Oct — the brain

- ☐ Snowpark Python procs: `check_waiting_period`, `apply_room_rent_proportion`, `apply_sub_limits`, `sla_status`
- ☐ Create the Cortex Agent: semantic view + search + four tools; write orchestration instructions that force a structured JSON decision with a citation per line
- ☐ Run 10 golden cases; fix tool descriptions until routing is right

Saturday 3 Oct — product and trust

- ☐ Streamlit console (container runtime): request queue with SLA timer, decision card with clickable citations, approve/edit, decision log
- ☐ Masking policy on clinical notes + row access policy by TPA; three demo users
- ☐ Task + Alert that notifies when a request passes 45 minutes
- ☐ `policy-onboarding` CoCo skill (`SKILL.md`) using the `skill-development` skill

Sunday 4 Oct — prove it and ship

- ☐ Run all 30 golden cases; show the accuracy scorecard in the app
- ☐ README: architecture, setup script, dataset list with licenses (Synthea Apache-2.0, policy wordings sources), `docs/coco-playbook.md` with the prompts you used
- ☐ Dedicated judge user; test login from a private browser window
- ☐ 10-slide deck: problem + 1-hour clock, demo flow, architecture, differentiators, accuracy, cost, roadmap
- ☐ Submit by 6 PM IST

Cut line if you fall behind (drop in this order): the SLA alert, the live second-policy onboarding (keep the skill in the repo), the Java UDF, the TPA row-access policy. Never cut the citations, the deterministic tools, or the golden-set scorecard — those are the win.

Spending the \$400 credit

\$400 is plenty for this MVP **if you control model choice and idle services**; it is easy to burn if you run a frontier model over bulk rows. Also keep a reserve — the finale runs 27–30 Oct, and §4.6 of the rules says finalists may get a fresh trial link only if the old one has expired.

Cost driver	How it bills	Guardrail
AI functions, Agents, CoCo, CoWork	AI Credits per million tokens, rate set by model (Snowflake AI pricing); AI Credits are about \$2.00 each with cross-region routing (Finout)	Small or mid-size model for bulk <code>AI_EXTRACT</code> ; a stronger model only for the agent. In CoCo, use <i>Auto Efficient</i> for routine work and <code>--mode code</code> for pure coding
Model spread	Per-model rates run from roughly \$0.12 to \$5.10 per million tokens (Finout)	Never run the top model over whole tables; test prompts on 5 rows first
Warehouse compute	Platform credits per running hour	XS warehouse, <code>AUTO_SUSPEND = 60</code> ; Snowflake recommends no larger than MEDIUM for AI functions (docs)
Cortex Search	Charged by index size and how long the index is kept, even when idle (Cortex Agents docs)	Keep the corpus to the policy clauses (a few MB); suspend or drop between work sessions
Streamlit container runtime	Compute pool time while the app runs	Stop the app when you are not demoing

Set these on day 0

1. A resource monitor on the account that notifies at 50% and 75% and suspends at 90% of your budget.
2. `AUTO_SUSPEND = 60` on every warehouse.

3. A daily check of spend: `SNOWFLAKE.ACCOUNT_USAGE.METERING_HISTORY` (AI services appear there) — or just ask CoCo's `cost-intelligence` skill "what did I spend today and on what?"
4. Resource budgets for Cortex Agents exist (added March 2026) — set one on your agent.

Learning path (learn while you build)

Don't study first — each day's build plan pulls in exactly one new service. Read the matching resource the morning you need it (about an hour each).

Day	Learn	Resource
Tue	CoCo CLI basics, skills, MCP	CoCo CLI docs · Bundled skills · Extensibility · hackathon workshop recordings (on demand)
Wed	Stages, loading, Snowpark basics	Snowpark Python guide · Synthea
Thu	AI functions, Cortex Search, semantic views	Cortex AI Functions · Semantic views overview · Semantic views and agents with Cortex Code (Medium)
Fri	Cortex Agents: tools, instructions, run API	Get started with Cortex Agents · Cortex Agents overview · Agents API + React quickstart (for citation handling)
Sat	Streamlit in Snowflake, masking, alerts	Ask CoCo to load <code>developing-with-streamlit</code> , <code>data-governance</code> and <code>alert</code> skills and explain as it builds
Sun	Agent evaluation, cost check	<code>agent-studio</code> evaluation workflow · AI pricing

Best learning trick with CoCo: ask it to explain before it acts — "Plan how you'd build a Cortex Search service over these clauses, explain each object you'll create, then wait." Use plan mode. You learn the service and keep control of your credits.

Sources

- [Hackathon page](#) · [Official Terms](#) · [YourStory announcement](#)
- [Snowflake CoCo overview](#) · [CoCo CLI](#) · [Code mode](#) · [Plugins](#) · [Atlan: CoCo explainer](#) · [Atlan: Cortex Sense](#)

- [Cortex Analyst → Agents transition note](#) · [AI function cost considerations](#) · [Finout: Cortex pricing 2026](#)
- IRDAI cashless timelines: [Life Insurance International](#) · [RTI wiki](#) · [Arogya Sanjeevani sample wording \(IRDAI\)](#)
- Competitor repos: listed in the problem-statement comparison above.